

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 2
Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Navya Maheshwari – 24011101071
AI & DS – B1

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Source Code

<https://github.com/nzoopxw/deep-learning-lab-sem5/tree/main/Lab-02>

5. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

6. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

7. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
 - Loss: Categorical Cross Entropy
 - Metric: Accuracy
- Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

8. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

9. Mandatory Plots

Generate the following plots.

1. Sample Images
2. Class Distribution
3. Training Accuracy vs Epoch
4. Validation Accuracy vs Epoch

5. Training Loss vs Epoch
6. Validation Loss vs Epoch
7. Confusion Matrix
8. Hyperparameter Search Results
9. Best Model Accuracy Comparison

10. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	64
Optimizer	RMSProp
Activation Function	Tanh
Epochs	10 (search) / 20 (retrain)
Dropout	0.2
Cross-validation Accuracy	0.8543
Testing Accuracy	0.8770

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8695	0.8770
Precision	0.8742	0.8788
Recall	0.8695	0.8770
F1-score	0.8704	0.8756
Training Time	115.6 s	67.0 s

Task-wise Outputs and Plots with Inference

Task 1: Dataset Exploration

The dataset was loaded and its dimensions printed. Ten sample images were displayed and the class distribution was plotted.

```
Training images : (60000, 28, 28)
Training labels : (60000,)
Testing images  : (10000, 28, 28)
Testing labels  : (10000,)
Pixel range     : 0 to 255
Number of classes: 10
```

Figure 1: Dataset summary (printed dimensions).

Inference: The dataset loads as 60,000 training and 10,000 test images, each a 28×28 grayscale matrix across 10 classes. These dimensions confirm the data is ready for flattening into 784-dimensional input vectors.

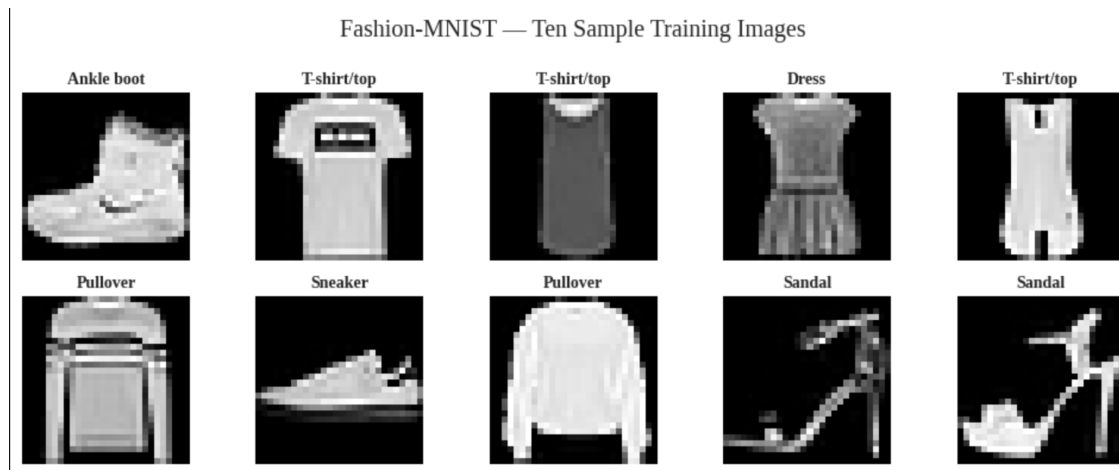


Figure 2: Ten sample images with class labels.

Inference: The samples show clear visual overlap among the upper-body garments (T-shirt, Pullover, Coat, Shirt), which share dark rectangular silhouettes. This similarity foreshadows the classes the model later finds hardest to separate.



Figure 3: Class distribution across the ten categories.

Inference: Every class contains exactly 6,000 training images, so the dataset is perfectly balanced. No class weighting or resampling is needed, and overall accuracy is a fair headline metric since no class dominates.

Task 2: Data Preprocessing

Images were flattened and normalized, and the tensor shapes were printed before and after preprocessing.

```
Before preprocessing:
x_train: (60000, 28, 28) | dtype: uint8
x_test : (10000, 28, 28) | dtype: uint8
y_train: (60000,) | dtype: uint8
Pixel range: 0 to 255

After preprocessing:
x_train: (60000, 784) | dtype: float32
x_test : (10000, 784) | dtype: float32
y_train: (60000, 10) (one-hot)
Pixel range: 0.0 to 1.0
```

Figure 4: Tensor shapes before and after preprocessing.

Inference: The training images change from (60000, 28, 28) to (60000, 784) and the labels from (60000,) to (60000, 10), while pixel values are rescaled from 0–255 to 0.0–1.0. This confirms the data is correctly flattened, normalized, and one-hot encoded for the dense network.

Task 3: Model Construction

The baseline MLP was constructed and its summary displayed, showing 109,386 trainable parameters.

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

Figure 5: `model.summary()` of the baseline MLP.

Inference: The model has 109,386 trainable parameters, of which 100,480 belong to the first dense layer connecting the 784 inputs to 128 units. The parameter count is dominated by this input layer, which is typical for an MLP applied to flattened images.

Task 4: Model Training

The model was compiled with Adam, categorical cross-entropy, and accuracy, then trained for 20 epochs at batch size 32.

```

Train split: (50000, 784) | Validation split: (10000, 784)

Epoch 1/20
1563/1563 - 16s - 10ms/step - accuracy: 0.8196 - loss: 0.5046 - val_accuracy: 0.8501 - val_loss: 0.4052
Epoch 2/20
1563/1563 - 10s - 7ms/step - accuracy: 0.8614 - loss: 0.3746 - val_accuracy: 0.8650 - val_loss: 0.3692
Epoch 3/20
1563/1563 - 5s - 3ms/step - accuracy: 0.8767 - loss: 0.3350 - val_accuracy: 0.8683 - val_loss: 0.3558
Epoch 4/20
1563/1563 - 4s - 3ms/step - accuracy: 0.8841 - loss: 0.3094 - val_accuracy: 0.8734 - val_loss: 0.3450
Epoch 5/20
1563/1563 - 4s - 3ms/step - accuracy: 0.8908 - loss: 0.2912 - val_accuracy: 0.8773 - val_loss: 0.3304
Epoch 6/20
1563/1563 - 5s - 3ms/step - accuracy: 0.8963 - loss: 0.2741 - val_accuracy: 0.8781 - val_loss: 0.3291
Epoch 7/20
1563/1563 - 4s - 3ms/step - accuracy: 0.9015 - loss: 0.2615 - val_accuracy: 0.8804 - val_loss: 0.3316
Epoch 8/20
1563/1563 - 5s - 3ms/step - accuracy: 0.9059 - loss: 0.2508 - val_accuracy: 0.8775 - val_loss: 0.3374
Epoch 9/20
1563/1563 - 4s - 3ms/step - accuracy: 0.9109 - loss: 0.2391 - val_accuracy: 0.8806 - val_loss: 0.3376
Epoch 10/20
1563/1563 - 4s - 3ms/step - accuracy: 0.9132 - loss: 0.2304 - val_accuracy: 0.8828 - val_loss: 0.3419
Epoch 11/20
1563/1563 - 5s - 3ms/step - accuracy: 0.9169 - loss: 0.2200 - val_accuracy: 0.8784 - val_loss: 0.3553
Epoch 12/20
...
Epoch 20/20
1563/1563 - 7s - 5ms/step - accuracy: 0.9369 - loss: 0.1695 - val_accuracy: 0.8766 - val_loss: 0.4080

Baseline training time: 115.6 s
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Figure 6: Training log across 20 epochs.

Inference: Across the 20 epochs, training accuracy rises steadily toward roughly 0.90 while validation accuracy plateaus near 0.88. The steady decrease in loss confirms stable convergence with the Adam optimizer.

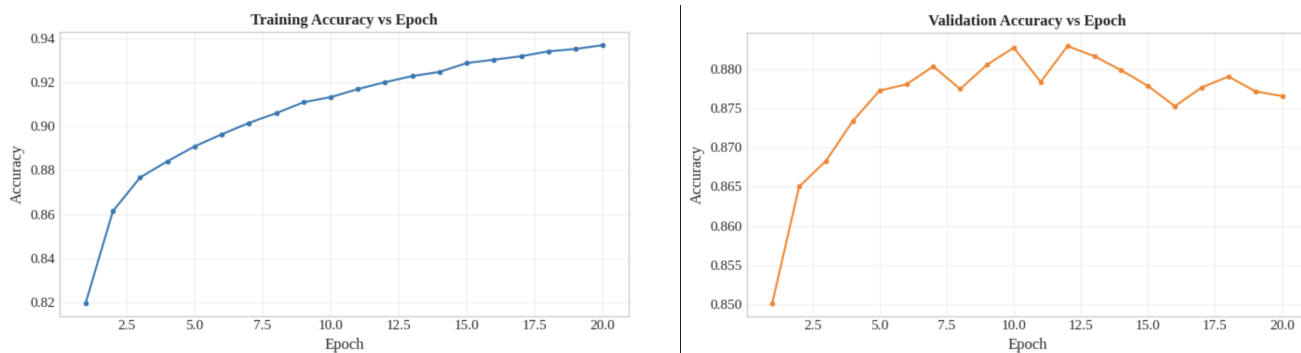


Figure 7: Training accuracy (left) and validation accuracy (right) versus epoch.

Inference: Training accuracy climbs smoothly and monotonically, whereas validation accuracy flattens near 0.88. The growing gap between the two curves is a mild sign of overfitting, which motivates using dropout during the hyperparameter search.

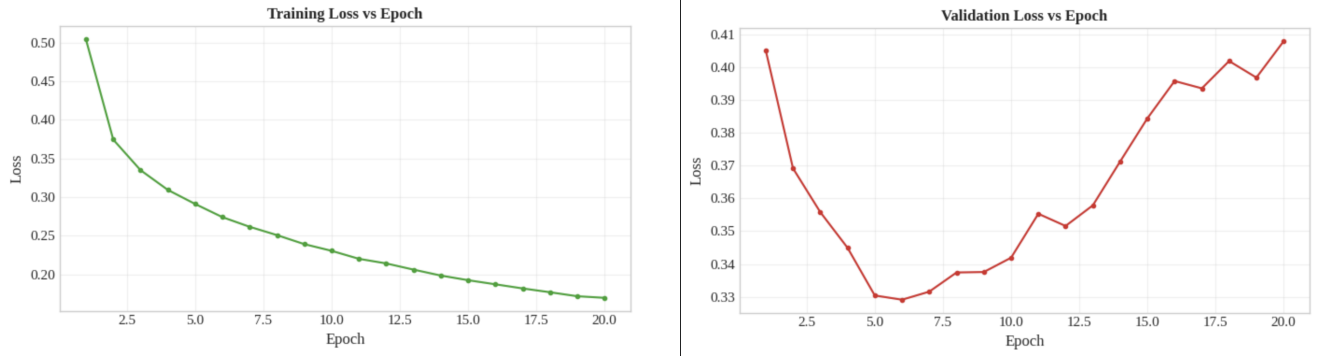


Figure 8: Training loss (left) and validation loss (right) versus epoch.

Inference: Training loss decreases consistently without oscillation, confirming a well-chosen learning rate. Validation loss falls initially and then flattens with small fluctuations, indicating the model stops generalizing further while it keeps fitting the training set.

Task 5: Model Evaluation

The trained model was evaluated on the test set, producing the metrics, classification report, and confusion matrix below.

===== BASELINE MODEL =====				
Accuracy	:	0.8695		
Precision	:	0.8742		
Recall	:	0.8695		
F1	:	0.8704		
Classification Report:				
	precision	recall	f1-score	support
T-shirt/top	0.8659	0.7490	0.8032	1000
Trouser	0.9888	0.9710	0.9798	1000
Pullover	0.7951	0.7450	0.7692	1000
Dress	0.9146	0.8250	0.8675	1000
Coat	0.7586	0.8170	0.7867	1000
Sandal	0.9446	0.9720	0.9581	1000
Shirt	0.6429	0.7560	0.6949	1000
Sneaker	0.9354	0.9410	0.9382	1000
Bag	0.9271	0.9790	0.9523	1000
Ankle boot	0.9691	0.9400	0.9543	1000
accuracy			0.8695	10000
macro avg	0.8742	0.8695	0.8704	10000
weighted avg	0.8742	0.8695	0.8704	10000

Figure 9: Accuracy, precision, recall, F1-score, and classification report.

Inference: The baseline model reaches a test accuracy of 0.8695 with macro-averaged precision, recall, and F1 all close to 0.87. The per-class report shows Shirt as the weakest class, consistent with the visual overlap seen in the sample images.

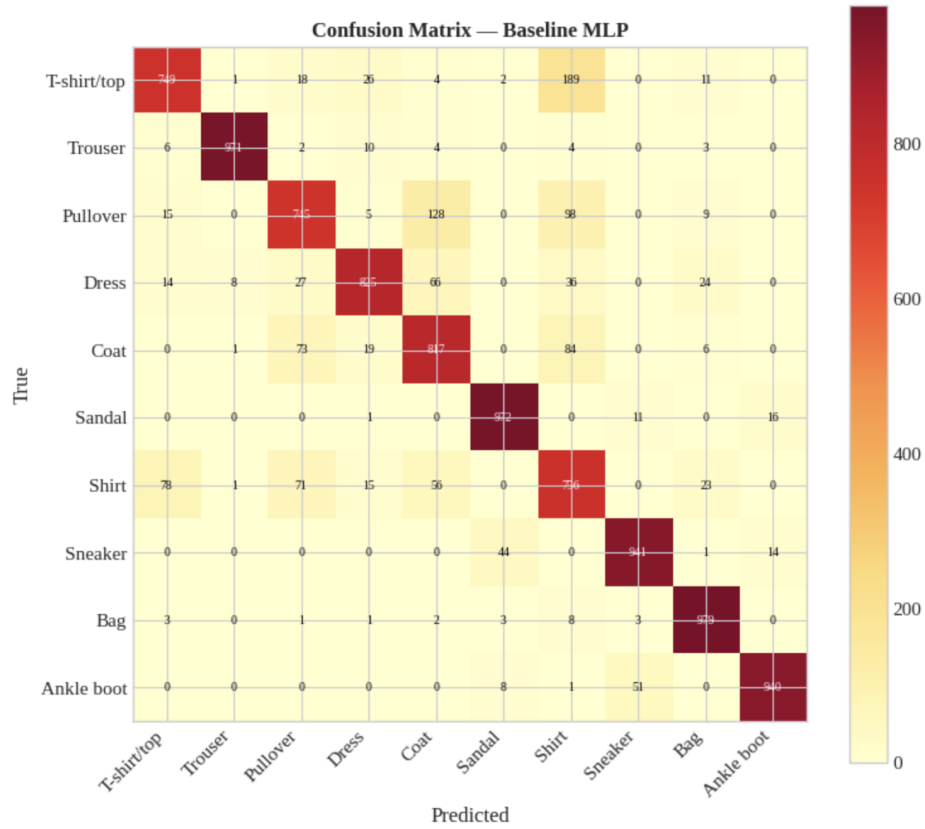


Figure 10: Confusion matrix of the baseline model.

Inference: The strongest off-diagonal confusions occur among Shirt, T-shirt/top, Pullover, and Coat. Shirt is most often misclassified as T-shirt or Coat, while Trouser, Bag, and the footwear classes are classified almost perfectly.

Hyperparameter Optimization Output

The randomized search results and the baseline-versus-optimized comparison are shown below.

```

Best 5-fold CV accuracy: 0.8543

Best hyperparameters:
  activation      : tanh
  batch_size     : 64
  dropout        : 0.2
  epochs         : 10
  learning_rate  : 0.001
  n_layers       : 1
  n_neurons      : 256
  optimizer      : rmsprop

```

Figure 11: Best hyperparameter combination and cross-validation accuracy.

Inference: The search identifies a single 256-unit tanh layer with RMSProp, learning rate 0.001, batch size 64, and dropout 0.2 as the best configuration, at a cross-validation accuracy of 0.8543. A single wide layer is sufficient because an MLP saturates on this dataset.

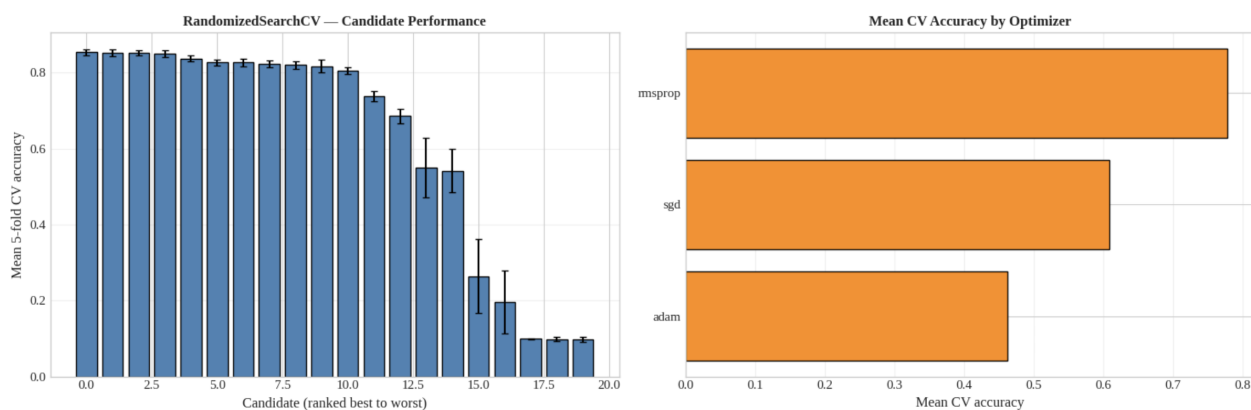


Figure 12: Hyperparameter search results.

Inference: The 20 sampled configurations span a wide accuracy range, from above 0.85 down to below 0.30. The poorest candidates are almost all those paired with a learning rate of 0.1, whose updates are too large to converge within ten epochs.

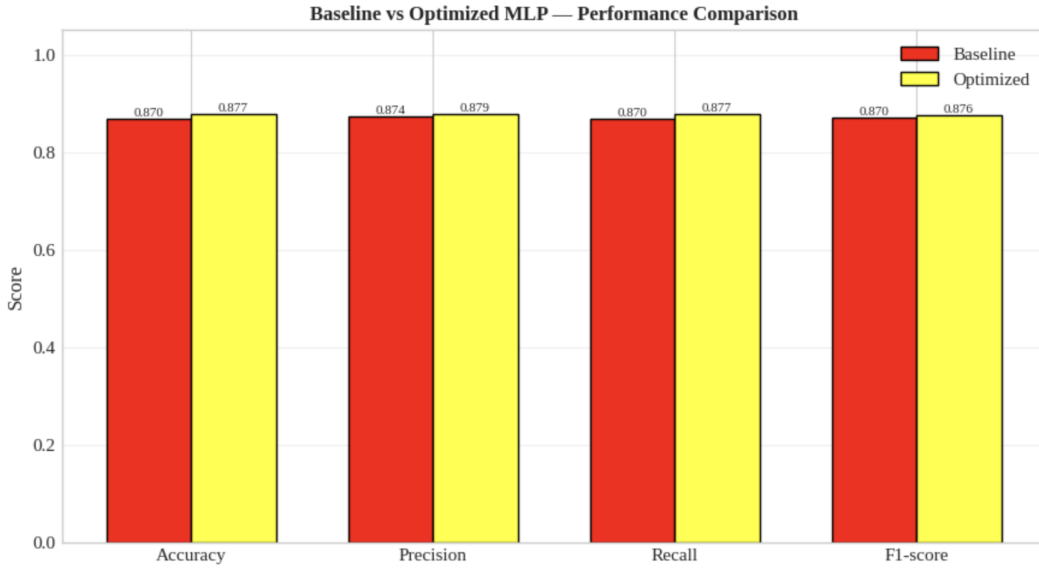


Figure 13: Best model accuracy comparison (baseline vs optimized).

Inference: The optimized model edges out the baseline on every metric (for example accuracy 0.8770 versus 0.8695). The improvement is small because MLP accuracy on Fashion-MNIST saturates near 88–89%; the ceiling is architectural rather than a matter of tuning.

11. Discussion

1. Which optimization method was used?

Automated hyperparameter optimization was performed using RandomizedSearchCV with 5-fold cross-validation, wrapping the Keras model through the SciKeras KerasClassifier. Randomized search was chosen over grid search because the full search space contained several thousand combinations, which is infeasible to evaluate exhaustively under cross-validation.

2. Which hyperparameters were selected?

The best configuration found was: 1 hidden layer, 256 neurons, tanh activation, RMSProp optimizer, learning rate 0.001, batch size 64, and dropout 0.2. This gave a cross-validation accuracy of 0.8543 and a retrained test accuracy of 0.8770.

3. Did optimization improve performance?

Yes, marginally. The optimized model improved test accuracy from 0.8695 to 0.8770 and F1-score from 0.8704 to 0.8756, while also reducing training time from 115.6 s to 67.0 s. The gain is small because a plain MLP on Fashion-MNIST saturates around 88–89%; the accuracy ceiling is set by the fully-connected architecture, so tuning yields only limited improvement.

4. Which hyperparameter had the greatest impact?

The learning rate had by far the largest effect. Averaged across all sampled configurations, a learning rate of 0.001 gave a mean CV accuracy of 0.825, dropping to 0.785 at 0.01 and collapsing to 0.285 at 0.1. A learning rate that is too large prevents the network from converging within the training epochs, so learning rate is the dominant factor determining whether the model learns at all.

5. Compare Grid Search and Randomized Search.

Grid search evaluates every combination in the search space exhaustively, guaranteeing the best grid point

is found but scaling multiplicatively with the number of hyperparameters, which becomes intractable for large spaces. Randomized search instead samples a fixed number of combinations at random, giving a controllable compute budget while still exploring the whole space, and it usually finds a near-optimal configuration at a fraction of the cost. For a search space of this size, randomized search is the practical choice.

6. Recommend the best MLP configuration.

For Fashion-MNIST with a plain MLP, the recommended configuration is a single wide hidden layer of 256 units with tanh activation, RMSProp at a learning rate of 0.001, batch size 64, and a dropout of 0.2, trained for about 20 epochs. This matches or exceeds the deeper baseline while training faster and generalizing slightly better. Beyond this, further accuracy gains would require a convolutional architecture rather than additional MLP tuning.

12. Conclusion

In this experiment, a Multi-Layer Perceptron was successfully implemented, trained, and evaluated on the Fashion-MNIST dataset. After preprocessing the images into normalized 784-dimensional vectors and one-hot encoding the labels, a baseline MLP with two hidden layers achieved a test accuracy of 86.95%. Automated hyperparameter optimization using 5-fold RandomizedSearchCV then identified a lighter single-layer configuration that raised the test accuracy to 87.70% while nearly halving the training time. The analysis showed that the learning rate is the single most influential hyperparameter, and that the model's accuracy ceiling near 88–89% is imposed by the fully-connected architecture rather than by the choice of hyperparameters. Overall, the experiment demonstrates the full workflow of building, evaluating, and systematically tuning an MLP, and highlights that moving to a convolutional network would be the natural next step for higher accuracy.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.